

MedExRAG System Design Guide

Medical Expert X-ray RAG Analysis | Version 1.0 | February 2026

1. System Overview

MedExRAG combines Vision Language Models (VLMs) with Retrieval-Augmented Generation (RAG) to deliver evidence-based radiology interpretation. The system ingests medical literature into a vector database, analyzes X-ray images with a VLM, and augments findings with retrieved literature context. A LangGraph multi-agent pipeline orchestrates the end-to-end diagnostic workflow.

Architecture layers (top to bottom):

- **User Interfaces** – Streamlit web app (port 8501) and CLI for scripting
- **Multi-Agent Orchestration** – LangGraph state machine: Analyst > Researcher > Diagnostician > Reporter
- **Core Components** – DocLing (PDF processing), ChromaDB + PubMedBERT (vector storage), Qwen2-VL-2B (vision-language model)
- **Observability** – LangSmith, Prometheus, OpenTelemetry/Jaeger, Grafana

Request flow: User submits X-ray > Analyst Agent generates initial findings via VLM > Researcher Agent queries vector store for relevant literature > Diagnostician Agent synthesizes findings with evidence > Reporter Agent compiles structured report with citations and confidence scores.

2. Key Technology Choices

2.1 Vision-Language Model: Qwen2-VL-2B-Instruct

Criterion	Qwen2-VL-2B (Selected)	LLaVA-1.5 (13B)	GPT-4V (API)
Model Size	4 GB	13 GB	Cloud-hosted
GPU Latency	2–5 s	5–10 s	~5 s
Cost per Image	\$0 (self-hosted)	\$0 (self-hosted)	~\$0.01
Data Privacy	Full control	Full control	Data leaves site
Reproducibility	Deterministic	Deterministic	Non-deterministic

Why Qwen2-VL-2B: At 4 GB it runs on commodity hardware with 2–5 s GPU latency. Self-hosting keeps patient data on-premises. Deterministic generation (`do_sample=False`) guarantees reproducible outputs for audit trails. Larger models (7B+) offer marginally better output but at unacceptable latency/memory cost.

2.2 Embeddings: PubMedBERT

Model	Training Data	Medical NER F1	Vocabulary
PubMedBERT	3.1B PubMed words	~92%	Biomedical
Generic BERT	Wikipedia + Books	~78%	General
SciBERT	1.2B science papers	~89%	Scientific

Why PubMedBERT: The 14-point F1 improvement over generic BERT translates directly to fewer false-positive literature matches and better understanding of medical terminology.

2.3 Vector Store: ChromaDB

Feature	ChromaDB (Selected)	Pinecone	FAISS
Deployment	Local, embedded	Cloud API	In-memory
Persistence	On-disk	Managed	File export
Infrastructure	None required	API key	None
Cost	Free	Per-query	Free

Why ChromaDB: Embedded persistent storage with HNSW-indexed cosine similarity search eliminates external dependencies. No data leaves the deployment boundary, simplifying HIPAA compliance. Single-machine is sufficient for medical KBs under 10 GB.

2.4 Document Processing: DocLing

Feature	DocLing (Selected)	PyPDF2	pdfplumber
Structure Preserving	Yes	No	Partial
Table Extraction	Yes	No	Rough
OCR Support	Yes	No	No

Why DocLing: Preserves document structure and exports clean markdown, producing higher-quality chunks. Built-in OCR handles scanned reports. Slower (~500 ms/page) but acceptable for batch ingestion.

2.5 Agent Orchestration: LangGraph

Pattern	LangGraph (Selected)	ReAct Agent	Simple Chain
Workflow Control	Explicit DAG	LLM-driven	Sequential
State Management	Typed, inspectable	Hidden	Minimal
Auditability	Full state trace	Prompt logs	Limited
Extensibility	Add/remove nodes	Modify prompt	Rigid

Why LangGraph: Medical workflows require explicit, auditable state transitions. The typed `AgentState` makes every agent’s I/O inspectable—essential for clinical audit trails. Adding agents requires only adding a graph node. Manual agent logic is justified by the control needed in a medical context.

3. RAG Pipeline Design

Two-Stage Analysis

The pipeline separates initial image interpretation from evidence-based augmentation:

Stage 1 – Initial Analysis: The VLM examines the X-ray without external context and generates preliminary findings (200–500 tokens).

Stage 2 – Literature-Enhanced Analysis: Preliminary findings become the semantic search query. Top-5 literature chunks are retrieved and formatted as context. The VLM generates a second analysis with literature evidence, citations, and a confidence score ($0.5 + \text{avg_relevance} * 0.4$, range 0.5–0.95).

Chunking Strategy

Parameter	Value	Rationale
Chunk size	800 chars	Fits a typical medical abstract paragraph
Overlap	150 chars	Preserves context across boundaries
Separators	<code>\n\n, \n, .,</code>	Respects paragraph > sentence > word hierarchy

This prevents splitting medical concepts mid-thought while keeping chunks small enough for precise retrieval.

4. Multi-Agent Architecture

Agent Pipeline

Four agents execute in a linear workflow, each building on the previous output:

Agent	Tool	Input	Output
Analyst	ImageAnalysisTool	X-ray image	Initial radiological findings
Researcher	LiteratureSearchTool	Initial findings (as query)	Top-5 literature chunks with relevance scores
Diagnostician	DiagnosticReasoningTool	Findings + literature	Differential diagnoses + recommendations
Reporter	Direct LLM generation	All accumulated state	Structured radiology report with citations

Shared State

All agents share state via a typed dictionary:

```
class AgentState(TypedDict):  
    messages: List[BaseMessage]      # Accumulating history  
    image: Optional[Image.Image]      # Input X-ray
```

```

initial_findings: Optional[str]    # Analyst output
literature_context: Optional[str]  # Researcher output
diagnostic_reasoning: Optional[str] # Diagnostician output
final_report: Optional[str]        # Reporter output

```

Each agent reads upstream fields and writes to its own, creating clear data lineage. A custom LangChain LLM wrapper (NVReasonCXRLLM) adapts the VLM for agent tool-calling, so one model serves as both vision analyzer and reasoning backbone.

5. Observability & Deployment

Four-Tier Observability

Tier	Component	Purpose	Endpoint
M1	Structured Logging + LangSmith	JSON logs with correlation IDs; LLM call tracing	LangSmith cloud
M2	Prometheus	Latency histograms, counters, gauges	:8002/metrics
M3	OpenTelemetry + Jaeger	Distributed tracing across pipeline stages	:16686
M4	Grafana	Dashboards, alerting rules, health checks	:3000

Key metrics: VLM inference latency, vector search latency, end-to-end pipeline latency, per-agent step duration, vector store chunk count. **Alerts:** VLM P95 > 30s, error rate > 10%, empty vector store. **Graceful degradation:** All observability is optional—core pipeline continues if monitoring services are unavailable.

Docker Deployment

Six services orchestrated via Docker Compose:

Service	Port	Memory	Purpose
streamlit	8501	6–12 GB	Web UI + VLM inference
cli-worker	–	4–8 GB	Batch processing (on-demand)
prometheus	9090	512 MB	Metrics collection
otel-collector	4317	256 MB	Trace collection
jaeger	16686	512 MB	Trace visualization
grafana	3000	512 MB	Dashboards

Build strategy: Multi-stage Dockerfiles with a shared base image (Python 3.11). **GPU support:** Overlay compose file adds NVIDIA GPU reservations; auto-detects CUDA with CPU fallback. **Model caching:** HuggingFace models (~5 GB) persisted in a named volume across restarts.

Evaluation Strategy

Three-level evaluation: (1) **Component** – VLM quality via LLM judge, vector search precision/recall; (2) **RAG Quality** – NDCG@5, MRR, citation accuracy; (3) **End-to-End** – functional tests, performance benchmarks (target: <30s with RAG), robustness tests. DVC pipelines automate evaluation runs and track metrics over time.

Summary of Design Principles

Principle	Implementation
Privacy-first	All components self-hosted; no external API dependencies
Medical determinism	<code>do_sample=False</code> ; reproducible outputs for audit trails
Domain-specific AI	PubMedBERT embeddings; biomedical vocabulary understanding
Explicit orchestration	LangGraph DAG with typed state; inspectable transitions
Graceful degradation	Observability optional; core pipeline always functional
Structure preservation	DocLing maintains document hierarchy for quality chunking
Production readiness	Docker-native; GPU support; health checks; alerting